CortexAI Redis Cache & Session Management Documentation

This document explains the setup, configuration, variables, functions, and the detailed file-to-file state flow showing how Redis caches and validates data across CortexAI.

---

1. Setup & Connection Configuration

Redis is configured as an in-memory database to store session data.

A. Infrastructure Setup

- **File**: `docker-compose.yml`
- **What it does**: Spins up a Docker container running Redis.
- **How it works**: Exposes Redis on standard port `6379`, mapping it locally.

B. Central Connection Client

- **File**: `redis.js` (inside `shared/redis/`)
- **What it does**: Initializes the Redis database driver.
- **How it works**:
  - Imports `ioredis`.
  - Creates a single exported connection instance `redis` using `new ioredis(process.env.REDIS_URL)`.
  - Listens to the `connect` event: `redis.on("connect", ...)` to print a confirmation message in logs.

---

2. Variables & Functions Dictionary

Shared Client

- `redis` (instance in `redis.js`):
  - **What it is**: The exported database client used by other backend modules to perform operations.

Auth Service (Port 8001)

- `login` (controller in `auth.controller.js`):
  - **Variables**:
    - `sessionid`: A cryptographically secure random UUID generated via `crypto.randomUUID()`.
    - `key`: The string `session-${sessionid}` used to index the session in Redis.
    - `payload`: A stringified JSON representation of user data `{ userid, name, email, avatar }`.
  - **Redis Command**: `await redis.set(key, payload, "EX", 24 * 60 * 60 * 7)`. Caches the session details, setting an automatic expiration of 7 days.
- `logout` (controller in `auth.controller.js`):
  - **Variables**:
    - `sessionid`: Cookie identifier retrieved from `req.cookies.session`.
  - **Redis Command**: `await redis.del("session-" + sessionid)`. Instantly deletes the cache key, logging the user out.

Gateway (Port 8000)

- `protect` (middleware in `auth.middleware.js`):
  - **Variables**:
    - `sessionId`: The cookie value sent from the client's browser.
    - `session`: The stringified user profile returned from Redis.
  - **Redis Command**: `await redis.get("session-" + sessionId)`. Fetches the cache to verify if the session is active.

---

3. Global Integration (Where, When, & How Redis is Imported and Used)

Redis acts as a shared data store between different servers. Below is where and when it is imported and utilized across the project files:

A. Auth Service: Session Writes & Invalidation (`auth.controller.js`)

- **Where**: Imported as `import redis from "../../../shared/redis/redis.js"`.
- **Writing Sessions (Login)**:
  - *When*: Triggered immediately after Google Token verification and MongoDB synchronization.
  - *How*: Calls `redis.set()` to cache the user profiles in memory. This eliminates the need for downstream services to hit MongoDB to authenticate subsequent requests.
- **Wiping Sessions (Logout)**:
  - *When*: Triggered when a user clicks the logout button.
  - *How*: Calls `redis.del()` using the session ID extracted from client cookies, invalidating the session immediately.

B. Gateway Service: Session Verification Middleware (`auth.middleware.js`)

- **Where**: Imported as `import redis from "../../shared/redis/redis.js"`.
- **Session Lookups**:
  - *When*: Triggered on every incoming HTTP request targeting protected routes (such as `/me`, `/chat/*`, or `/agent/*`).
  - *How*: The middleware calls `redis.get("session-" + sessionId)`. If the session key is present, it parses the JSON data and sets the `req.user` context. If the key is missing or expired, it immediately returns a `401 Unauthorized` status response, stopping further execution.

---

4. File-to-File Redis Execution Workflow

Below is the step-by-step path the code takes to write, read, and delete data in Redis:

Flow A: Writing Session Cache (On User Login)

```
[Trigger] auth.controller.js
   │  ──> User completes sign-in. Controller generates sessionId.
   ▼
[Client Config] shared/redis/redis.js
   │  ──> Imports client and issues command:
   │      redis.set("session-" + sessionid, JSON.stringify(userData), "EX", 604800)
   ▼
[Redis Cache] Docker Container (Port 6379)
      ──> Stores the session key with a 7-day expiration (TTL).
```

Flow B: Reading Session Cache (On Request Validation)

```
[Protected Request] gateway/index.js
   │  ──> User visits a protected page. Gateway routes requests through protect middleware
   ▼
[Middleware] auth.middleware.js
   │  ──> Extracts sessionId cookie.
   │  ──> Requests key: redis.get("session-" + sessionId) from shared/redis/redis.js
   ▼
[Redis Cache] Docker Container (Port 6379)
   │  ──> Returns the stringified user object from memory.
   ▼
[Context Setup] auth.middleware.js
      ──> Parses user details, binds to req.user context, and forwards request.
```

Flow C: Deleting Session Cache (On User Logout)

```
[Trigger] auth.controller.js
   │  ──> User clicks Logout. Controller extracts sessionId from browser cookies.
   ▼
[Client Config] shared/redis/redis.js
   │  ──> Imports client and issues command: redis.del("session-" + sessionid)
   ▼
[Redis Cache] Docker Container (Port 6379)
      ──> Deletes session key. Future checks on protect middleware now fail.
```